Du cadrage à l'interface

TP1, TP2, TP3 : construire une application de sentiment, de A à Z

A. Larhlimi

Développement et déploiement d'applications intelligentes
Trois séances de 2 h · travail en binôme · 12 octobre 2026

Ce document enchaîne les trois premières séances sur un seul et même dépôt. Il donne, pour chaque étape, son objectif, les commandes avec leur explication, le résultat attendu et les erreurs possibles. À la fin, votre binôme dispose d'une application complète : un modèle entraîné et évalué, et une interface qu'une personne non technique peut utiliser.

Table des matières

1 Vue d'ensemble **2**
1.1 Le besoin . . . 2
1.2 Les trois séances . . . 3
1.3 Les compétences visées . . . 3
1.4 Comment lire ce document . . . 3

2 Prérequis et réglages **3**
2.1 Ce qu'il faut installer . . . 3
2.2 Vérifier, puis régler . . . 4
2.3 Répartir les rôles . . . 4

3 Les ressources partagées **4**
3.1 Trois kits qui s'empilent . . . 4
3.2 Installer un kit . . . 5
3.3 L'entrepôt complet . . . 5

4 Architecture du dépôt **5**
4.1 Les sept dossiers . . . 5
4.2 requirements.txt . . . 6
4.3 .gitignore . . . 6
4.4 Les autres fichiers . . . 6

5 TP1 : cadrer et préparer l'environnement **7**
5.1 Étape 1. Décompresser le kit (5 min, chacun) . . . 7
5.2 Étape 2. L'environnement virtuel (10 min, chacun) . . . 7
5.3 Étape 3. Les trois fiches de cadrage (50 min, à deux) . . . 8
5.3.1 La grille et l'arbre . . . 8
5.3.2 Les trois cas . . . 8
5.3.3 Anatomie d'une fiche . . . 8
5.4 Étape 4. Créer le dépôt et pousser (25 min, membre A) . . . 9
5.4.1 Sur github.com . . . 9
5.4.2 Sur votre poste . . . 9
5.4.3 L'authentification . . . 10
5.5 Étape 5. L'installation complète (15 min, les deux) . . . 10
5.6 Étape 6. Un commit par membre (10 min, chacun) . . . 11
5.7 Étapes 7 et 8. Vérification et clone frais (15 min) . . . 11

6 TP2 : entraîner, évaluer, sauvegarder le modèle 12
6.1 Les quatre scripts du kit . . . 12
6.2 Étape A. Installer le kit (15 min) . . . 12
6.3 Étape B. Le jeu de démonstration (15 min) . . . 12
6.4 Étape C. Entraîner et sauvegarder (25 min) . . . 13
6.4.1 Les trois lignes de code à lire . . . 13
6.5 Étape D. Évaluer et comprendre les erreurs (30 min) . . . 14
6.6 Étape E. Le vrai jeu, les tests, le push (35 min) . . . 14

7 TP3 : l'interface de démonstration 15
7.1 Streamlit en trois idées . . . 15
7.2 Pourquoi deux fichiers . . . 15
7.3 Étape A. Installer et lancer (15 min) . . . 16
7.4 Étape B. L'onglet « un avis » (30 min) . . . 16
7.5 Étape C. L'onglet « un fichier CSV » (40 min) . . . 16
7.6 Étape D. Tests, documentation, push (35 min) . . . 16
7.7 Sur Google Colab . . . 17

8 Les 45 tests 17
8.1 Répartition . . . 17
8.2 Ce qui est ignoré, et pourquoi . . . 18
8.3 Les paramètres réglables . . . 18

9 Intégration continue et MLOps 18
9.1 Trois artefacts, pas un . . . 18
9.2 Ce que fait l'intégration continue . . . 19
9.3 Le geste MLOps de la séquence . . . 19
9.4 DevOps, MLOps : où passe la frontière . . . 20

10 Le compte rendu global 20

11 Récapitulatif 21
11.1 Les dix-sept contrôles . . . 21
11.2 Les erreurs les plus coûteuses . . . 21
11.3 La structure finale du dépôt . . . 22

1 Vue d'ensemble

1.1 Le besoin

Une équipe relation client reçoit des milliers d'avis par jour et les traite dans l'ordre d'arrivée. On veut faire remonter les **avis négatifs** en tête de file, pour répondre d'abord aux clients mécontents.

Le problème est volontairement simple côté modèle. Classer un avis en positif ou négatif se fait en vingt lignes de code, sans GPU, avec un classifieur linéaire. La difficulté est ailleurs, et c'est le sujet du module : rendre ce modèle reproductible, testé, surveillé, utilisable par quelqu'un d'autre, et redéployable après un réentraînement. Autrement dit, **l'écart entre un notebook et un produit**.

1.2 Les trois séances

Séance	Ce que le binôme produit	Contrôles	Tests
TP1	Dépôt structuré, environnement reproductible, trois fiches de cadrage	T0 à T7	38
TP2	Le modèle : entraînement, évaluation, métriques versionnées	C1 à C5	42
TP3	L'interface de démonstration Streamlit	S1 à S4	45

Un seul dépôt, `sentiment-app`, du premier jour au dernier. Chaque kit se pose sur le précédent sans rien écraser, et les tests s'accumulent au lieu de se remplacer.

1.3 Les compétences visées

Compétence	Vérifiée par
Choisir une approche technique et la justifier	12 tests de cadrage, lecture des fiches
Produire un modèle reproductible et l'évaluer correctement	4 tests de modèle, `metrics.json`
Exposer le modèle à un utilisateur non technique	3 tests d'interface
Travailler à deux sur un dépôt, avec tests et intégration continue	23 tests d'environnement, 3 d'historique

1.4 Comment lire ce document

Dix-sept contrôles numérotés jalonnent les trois séances : T0 à T7 pour le TP1, C1 à C5 pour le TP2, S1 à S4 pour le TP3. À chacun, le résultat attendu est écrit noir sur blanc.

Ne passez à l'étape suivante que lorsque le contrôle précédent a donné ce résultat. Deux d'entre eux doivent être rouges : c'est précisé à l'endroit voulu. Un binôme qui enchaîne sans vérifier arrive à la fin avec trois problèmes mélangés et ne sait plus lequel vient d'où.

2 Prérequis et réglages

Objectif

Arriver avec un poste prêt. Les quatre réglages de cette section suppriment à eux seuls trois des blocages les plus fréquents des séances.

2.1 Ce qu'il faut installer

Outil	Version	Remarque
Python	3.11.x (3.10 toléré)	Windows : cocher « Add python.exe to PATH ». **Évitez 3.12 et 3.13** : les versions figées du cours n'ont pas de paquet précompilé pour elles
Git	dernière stable	L'installeur Windows fournit aussi Git Bash, qui exécute les scripts `.sh`
VS Code	dernière	Facultatif, mais le kit contient sa configuration
Compte GitHub		Avec l'adresse de l'école, de préférence

2.2 Vérifier, puis régler

```
python --version                    # 3.11.x attendu ; sinon py -3.11 --version
git --version                       # si « commande inconnue », rouvrez le terminal
```

```
git config --global user.name "Prénom Nom"
git config --global user.email "prenom.nom@ecole.ma"
git config --global init.defaultBranch main
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Ces quatre lignes méritent une explication, car elles reviennent dans trois cas problèmes.

`user.name` est inscrit comme auteur dans chacun de vos commits. C'est ce nom, et non l'adresse, que compte le contrôle T5. Sans identité configurée, Git refuse de créer le commit avec `Please tell me who you are`.

`user.email` sert à GitHub pour rattacher le commit à votre compte. Si elle ne correspond à aucune adresse enregistrée dans votre profil, vos commits apparaissent sans votre photo ni votre lien, même s'ils sont bien de vous. Git ne vérifie jamais cette adresse : elle déclare qui vous dites être, pas qui vous êtes. L'authentification, elle, passe par le jeton d'accès personnel.

`-global` écrit dans votre fichier personnel, qui vaut pour tous les dépôts de ce compte utilisateur. La portée `-local`, elle, n'écrit que dans le `.git/config` du dépôt courant et l'emporte sur le global ; elle exige donc d'être *dans* un dépôt, sans quoi Git répond `-local can only be used inside a git repository`.

`Set-ExecutionPolicy` autorise l'exécution des scripts PowerShell pour votre seul compte. Sans elle, `Activate.ps1` est refusé et le venv ne s'active pas.

2.3 Répartir les rôles

Rôle	Ce qu'il fait seul	Ce qu'il fait comme l'autre
Membre A	Crée le dépôt GitHub, pousse le kit	venv, installation, son commit, vérifications
Membre B	Clone le dépôt une fois créé	venv, installation, son commit, vérifications

Décidez maintenant : les étapes 4 et 5 du TP1 en dépendent.

3 Les ressources partagées

3.1 Trois kits qui s'empilent

Archive	Contenu	Se pose sur
`sentiment-app.zip`	Le dépôt de départ complet : structure, scripts, 38 tests, CI, six documents	rien, c'est la base
`tp2-semaine2-kit.zip`	`src/` (quatre scripts), `tests/test_model.py`, notebook Colab	le dépôt du TP1
`tp3-streamlit-kit.zip`	`ui/` (interface et helpers), `tests/test_ui.py`, notebook Colab, documents du compte rendu	le dépôt après le TP2

3.2 Installer un kit

Décompressez l'archive **à côté** de votre dépôt, pas dedans. Vous devez obtenir deux dossiers voisins :

```
IA-DEV/
|-- sentiment-app/      <- votre dépôt
|-- tp2-semaine2/       <- le kit que vous venez de décompresser
```

```
cd tp2-semaine2
python appliquer_tp2.py ../sentiment-app --simulation   # voir sans rien écrire
python appliquer_tp2.py ../sentiment-app                # copier pour de bon
```

Le script vérifie d'abord que la cible ressemble bien au dépôt attendu, puis copie les fichiers un par un en annonçant pour chacun s'il l'ajoute ou le remplace. Celui du TP2 ajoute en outre une ligne au `.gitignore`, celui du TP3 refuse de s'installer si `src/predict.py` est absent, c'est-à-dire si le TP2 n'a pas été fait.

3.3 L'entrepôt complet

Dossier	Fichier	Pour qui	Quand
	`tp1-tp2-tp3.md` et ce PDF	Tous	La démarche complète
	`TP3-colab-streamlit.ipynb`	Étudiants	L'interface sur Colab
`kits/`	les trois archives	Étudiants	Une par séance
`compte-rendu/`	énoncé et modèle à remplir	Étudiants	Après le TP3
`diapositives/`	quatre PDF	Projection	Vue d'ensemble, TP1 détaillé, TP1 démarche, TP2
`enseignant/`	corrigés, QCM, guide	**Enseignant seulement**	Ne pas distribuer

Calendrier de diffusion conseillé : les prérequis une semaine avant, le kit de la séance deux jours avant, les diapositives le jour même, l'énoncé du compte rendu après le TP3.

4 Architecture du dépôt

Objectif

Savoir à quoi sert chaque fichier avant de l'utiliser. Un dépôt dont on ne comprend pas la structure se dégrade en trois semaines.

4.1 Les sept dossiers

Dossier	Contient	Se remplit
`data/`	Jeux de données	TP2, versionné par DVC en semaine 7
`models/`	Modèles entraînés	TP2. Jamais dans Git : lourd, reconstructible
`app/`	API FastAPI	Semaine suivante
`ui/`	Interface Streamlit	TP3
`tests/`	Tests pytest	Dès le TP1
`scripts/`	Installation et diagnostic	Fourni
`docs/cadrage/`	Fiches de cadrage	Partie A du TP1

Les dossiers encore vides contiennent un fichier `.gitkeep`, vide lui aussi. Raison : **Git ne versionne pas les dossiers, seulement les fichiers**. Sans ce témoin, un dossier vide disparaîtrait au clone, et la structure ne serait pas la même chez tout le monde.

4.2 requirements.txt

```
pandas==2.2.*
scikit-learn==1.5.*
joblib==1.4.*
pyarrow==21.0.*
fastapi==0.115.*
streamlit==1.39.*
mlflow==2.17.*
```

La syntaxe `==2.2.*` accepte 2.2.0 ou 2.2.3, jamais 2.3 : les correctifs passent, les changements de comportement non. Le test `test_requirements_pins_versions` vérifie que chaque ligne utile contient `==`.

Pourquoi installer tout le semestre dès la première séance ? Pour ne plus jamais y revenir. Une installation de dix minutes aujourd'hui vaut mieux que dix interruptions de deux minutes réparties sur onze semaines.

4.3 .gitignore

```
.venv/                      # l'environnement virtuel : des centaines de Mo
__pycache__/                # le bytecode, régénéré à chaque exécution
.env                        # les secrets : clés d'API, mots de passe
models/*                    # les modèles entraînés
!models/.gitkeep            # exception : le fichier témoin
!models/metrics.json        # exception : la mesure, ajoutée par le kit TP2
data/*                      # les jeux de données
!data/sample_reviews.csv    # exception : le petit jeu d'exemple
.vscode/*
!.vscode/settings.json      # exception : la configuration partagée
```

Le `!` inverse une règle. Cette mécanique d'exception est le cœur du dépôt : elle sépare ce qui est **lourd et reconstructible** (exclu) de ce qui est **léger et partagé** (inclus). La section 9 y revient, car c'est elle qui porte tout le raisonnement MLOps.

4.4 Les autres fichiers

Fichier	Rôle
`.env.example`	Modèle des variables secrètes, versionné car vide de vrais secrets. Chacun le copie en `.env`, lui-même ignoré
`pytest.ini`	`testpaths = tests` évite de parcourir tout le dépôt ; `addopts = -ra` résume échecs et tests ignorés avec leur raison
`.editorconfig`	UTF-8, fins de ligne Unix, indentation : évite les conflits illisibles entre Windows et macOS
`.vscode/settings.json`	Sélectionne l'interpréteur `.venv` et active pytest : supprime la cause la plus fréquente du `ModuleNotFoundError`
`Makefile`	Raccourcis `make setup`, `make check`, `make test` (Linux, macOS, Git Bash)
`README.md`	Porte d'entrée : installation en trois commandes, structure, tableau du binôme
`MARCHE-A-SUIVRE.md`	Le pas à pas de la séance
`TESTS.md`, `AIDE-GIT.md`, `DEPANNAGE.md`	Tests en détail, commandes Git, messages d'erreur

5 TP1 : cadrer et préparer l'environnement

Objectif

Décider quelle approche technique employer pour trois besoins métier, et monter un dépôt que l'enseignant peut cloner sur une machine vierge et faire tourner en trois commandes.

5.1 Étape 1. Décompresser le kit (5 min, chacun)

```
unzip sentiment-app.zip          # Windows : clic droit sur le zip, Extraire tout
cd sentiment-app                 # toutes les commandes du TP partent d'ici
ls
ls -Force | findstr gitignore    # Linux, macOS : ls -a | grep gitignore
```

Contrôle T0 : le kit est complet

Sept dossiers, six documents, `requirements.txt`, `pytest.ini`, et `.gitignore`, qui ne s'affiche pas avec un `ls` ordinaire parce qu'il commence par un point.

Pourquoi ce contrôle existe : `.gitignore` est le seul rempart contre l'erreur irréversible du contrôle T3. Son absence se repère maintenant, pas dans vingt minutes.

Si une commande dit plus tard qu'un fichier est introuvable, le premier réflexe est de taper `cd` sans argument pour voir où vous êtes. L'archive crée un dossier `sentiment-app` : un chemin du type `sentiment-app\sentiment -app` est normal, descendez jusqu'à celui qui contient `README.md`.

5.2 Étape 2. L'environnement virtuel (10 min, chacun)

Le Python du système est unique et partagé par tous vos projets. Les paquets du cours sont figés à des versions précises : les installer globalement casserait vos autres projets, et l'inverse aussi. Le venv est une copie légère de Python, propre au projet, avec son propre `pip` et son propre `site-packages`. Comme il se reconstruit en une commande, il ne part jamais dans Git.

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1          # Linux, macOS : source .venv/bin/activate
python -m pip install pandas scikit-learn pytest
python -m pytest -q
```

Commande	Ce qu'elle fait
`python -m venv .venv`	Exécute le module `venv` livré avec Python ; crée `.venv/`
`Activate.ps1`	Modifie le PATH du **terminal courant** : l'invite prend le préfixe `(.venv)`
`python -m pip install`	Passe par l'interpréteur courant, donc ne dépend pas du PATH

Trois paquets seulement ici, pour atteindre le premier contrôle en une minute. L'installation complète vient à l'étape 5.

T1 doit être ROUGE

Résultat attendu : **3 failed, 31 passed, 4 skipped**. Les trois échecs sont les fiches de cadrage vides, les quatre tests ignorés attendent un dépôt Git qui n'existe pas encore, et les 31 verts confirment que la structure, les paquets et le `.gitignore` sont en place.

Si vous obtenez `38 passed` ici, vous testez un autre dossier. Un test rouge avant que le travail soit fait est un test utile ; un test vert dès le départ ne vérifie rien.

Un test mérite d'être regardé : `test_virtualenv_active` compare `sys.prefix` et `sys.base_prefix`. Sur le Python du système, les deux sont égaux ; dans un venv, non. Il existe pour que l'oubli d'activation se voie tout de suite, au lieu de produire une erreur obscure trois commandes plus loin.

5.3 Étape 3. Les trois fiches de cadrage (50 min, à deux)

5.3.1 La grille et l'arbre

	Critère	La question à se poser
(a)	Données étiquetées	A-t-on des exemples annotés, ou peut-on en produire à faible coût ?
(b)	Vérifiabilité	La réponse doit-elle être tracée jusqu'à une source ou une règle ?
(c)	Coût par requête	Combien d'appels par jour, et que coûte chacun ?
(d)	Conséquence d'une erreur	Que se passe-t-il concrètement quand le système se trompe ?

Parcourez les questions dans l'ordre ; la première réponse « oui » donne l'approche.

1. Logique connue, stable, devant rester explicable ? → **règles métier**
2. Données étiquetées pour une tâche répétitive ? → **machine learning**
3. Réponse à tirer de documents existants, et à citer ? → **RAG**
4. Sinon, et seulement sinon → **modèle génératif seul**

On ne saute jamais directement à la dernière ligne : c'est l'option la plus coûteuse et la moins vérifiable.

5.3.2 Les trois cas

Fiche	Le cas	Réponse attendue
`cas1-pharmacie.md`	Ordonnances incomplètes	Règles métier : (a) pas d'étiquettes, (b) explicabilité exigée, (d) erreur grave
`cas2-reglement.md`	Règlement intérieur	RAG : (b) la réponse cite l'article, (a) pas de questions étiquetées, (c) volume faible
`cas3-avis-negatifs.md`	Avis clients négatifs	Machine learning : (a) étiquettes gratuites via la note, (c) milliers d'avis, (d) erreur peu grave

Douze minutes par cas. Un membre tient le clavier, l'autre parcourt l'arbre à voix haute et conteste ; inversez à chaque fiche.

Deux approches peuvent se défendre pour un même cas : c'est la justification qui est notée, pas la case cochée. Le machine learning reste acceptable pour la pharmacie si vous précisez qu'il faudrait d'abord constituer un jeu étiqueté et conserver la validation du pharmacien.

5.3.3 Anatomie d'une fiche

```
## Besoin en une phrase (obligatoire)

> Remplir chaque rubrique en une à trois phrases.     <- consigne, à laisser en
    place

Signaler au pharmacien toute ordonnance à laquelle il manque un champ obligatoire.
```

```
## Approche retenue (obligatoire)

- [x] Règles métier          <- une seule croix dans tout le fichier
- [ ] Machine learning
- [ ] RAG
- [ ] Modèle génératif seul

## Justification (obligatoire)

(a) Aucune ordonnance étiquetée n'existe au départ. (b) Le pharmacien doit pouvoir
expliquer le refus au patient. (d) Une ordonnance validée à tort a une conséquence
    grave.
```

Les autres rubriques obligatoires : utilisateur final, données nécessaires et leur origine, métrique de succès et seuil, conséquence d'une erreur et validation humaine, risques éthiques. La neuvième, facultative, est l'approche écartée et pourquoi.

Deux réflexes attendus : la métrique est chiffrée sur un jeu défini (« rappel d'au moins 0,90 sur la classe négative », pas « de bons résultats »), et dès que l'erreur a une conséquence réelle, vous écrivez qui valide et à quel moment.

Contrôle T2 : les fiches passent

`python -m pytest tests/test_cadrage.py -q` doit afficher **12 passed**. Les messages d'échec nomment la fiche et la rubrique fautives.

L'erreur qui coûte le plus de temps

Écrire « il y a des données étiquetées » **sans** la lettre `(a)` fait échouer le test. Les lettres doivent apparaître littéralement dans la justification, et il en faut au moins deux.

5.4 Étape 4. Créer le dépôt et pousser (25 min, membre A)

5.4.1 Sur github.com

1. **New repository** : nom `sentiment-app`, visibilité **Private**.
2. **Ne cochez rien** dans « Initialize this repository with » : ni README, ni `.gitignore`, ni licence. Le kit les fournit, et un dépôt pré-rempli provoquerait un conflit au premier push.
3. **Settings**, section Access, **Collaborators**, bouton **Add people** : cherchez par identifiant GitHub, ajoutez le membre B en **Write** et l'enseignant en **Read**.
4. Copiez l'URL HTTPS depuis le bouton vert **Code**.

Le rôle Write autorise le push, le rôle Read ne l'autorise pas, mais les deux permettent de cloner et de consulter les exécutions de la CI. L'enseignant peut donc noter sans jamais pouvoir modifier votre travail. **L'invitation doit être acceptée** : tant que le membre B ne clique pas sur le lien reçu par courriel, son clone et son push échouent avec `403 Forbidden`.

5.4.2 Sur votre poste

```
git init            # crée le dépôt local, c'est-à-dire le dossier caché .git/
git add -A          # met tout en attente, sauf ce qu'exclut .gitignore
git status          # CONTRÔLE T3 : lire la sortie AVANT de valider
```

T3 : le seul contrôle irréversible du TP

La sortie ne doit contenir **aucune ligne commençant par** `.venv/`. Si elle en contient, arrêtez-vous : le `.gitignore` manque ou a été renommé.

Une fois le commit poussé, le dépôt pèse 300 Mo, GitHub refuse les fichiers de plus de 100 Mo, et le test `test_venv_never_committed` reste rouge même après un `git rm -r -cached .venv`, parce qu'il lit **tout l'historique**. La seule correction est alors de refaire le dépôt.

Profitez-en pour repérer tout fichier parasite : une archive téléchargée, un PDF, une capture d'écran. Un `git rm -cached "le fichier"` et une ligne `*.zip` dans le `.gitignore` règlent le cas.

```
git commit -m "Semaine 1 : structure du depot et kit TP1"
git branch -M main                  # GitHub attend main ; -M force le renommage
git remote add origin https://github.com/votre-compte/sentiment-app.git
git remote -v                       # vérifiez : PAS de chevrons dans l'URL
git push -u origin main             # -u mémorise le lien : ensuite git push suffit
```

Un raccourci fourni enchaîne ces commandes et refuse de pousser si `.venv/` est en attente : `bash scripts/init_depot.sh <url>`.

5.4.3 L'authentification

Au premier push, Git demande un identifiant et un mot de passe. **Le mot de passe de votre compte GitHub est refusé** : il faut un jeton d'accès personnel. GitHub, photo de profil, Settings, **Developer settings**, Personal access tokens, **Tokens (classic)**, Generate new token (classic), portée `repo`. Copiez-le immédiatement, il ne s'affiche qu'une fois, et collez-le à la place du mot de passe.

Plus simple, si vous préférez éviter les jetons :

```
winget install --id GitHub.cli
gh auth login           # GitHub.com, HTTPS, authentification par navigateur
gh auth setup-git
```

Deux erreurs qui se ressemblent

`Password authentication is not supported` est un problème d'**authentification** : GitHub ne sait pas qui vous êtes. Réponse : le jeton.

`403 Forbidden` est un problème d'**autorisation** : GitHub le sait, mais vous n'avez pas le droit d'écrire. Réponse : l'invitation de collaborateur, envoyée et acceptée.

Lisez le mot qui suit le code d'erreur avant de réessayer.

5.5 Étape 5. L'installation complète (15 min, les deux)

Le membre B clone après avoir accepté l'invitation ; le membre A reste dans son dossier. Chacun lance ensuite le script de mise en place.

```
git clone https://github.com/votre-compte/sentiment-app.git
cd sentiment-app
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1     # Windows
bash scripts/setup.sh                                          # Linux, macOS, Git
    Bash
```

Le script enchaîne quatre étapes affichées à l'écran : vérifier Python, créer `.venv`, installer `requirements.txt` puis figer l'état exact dans `requirements.lock`, lancer le diagnostic. Ses premières lignes méritent un commentaire :

```
set -euo pipefail                       # s'arrête à la première erreur
cd "$(dirname "$0")/.."                 # remonte à la racine du dépôt, d'où qu'on
    le lance
PY=$(command -v python3.11 || command -v python3 || command -v python)
[ -d .venv ] || "$PY" -m venv .venv     # ne recrée pas le venv s'il existe
```

Fichier	Qui l'écrit	Contenu	Dans Git ?
`requirements.txt`	L'enseignante	`pandas==2.2.*`	Oui, c'est le contrat
`requirements.lock`	`pip freeze`, chez vous	`pandas==2.2.3`	Oui, par **un seul** membre

Le lock diffère légèrement d'un système à l'autre : si les deux membres le poussent, vous obtenez un conflit à chaque fois.

Contrôle T4 : le diagnostic

`python scripts/check_setup.py` doit afficher **0 erreur(s)**. Un ou deux avertissements sont normaux à ce stade : les fiches si la partie A n'est pas finie, et le fait qu'un seul membre ait commité.

Le diagnostic distingue trois niveaux. `OK` ne demande rien. `ATTENTION` ne bloque pas, mais doit disparaître avant la fin de la séance. `ERREUR` bloque : venv non activé, paquet absent, dossier manquant, pas de dépôt Git, `.venv/` suivi.

5.6 Étape 6. Un commit par membre (10 min, chacun)

```
git pull                          # récupère les commits de l'autre AVANT de pousser
# ajoutez votre ligne au tableau du binôme dans README.md
git add README.md                 # un seul fichier nommé, pas -A
git commit -m "README : ajout de <nom> au tableau du binome"
git push
```

Pourquoi nommer le fichier plutôt que `-A` : cela évite d'embarquer le `requirements.lock` régénéré sur votre poste, qui provoquerait un conflit chez l'autre membre. Les messages de commit sont notés : « README : ajout de Sara au tableau » plutôt que « update ».

En cas de conflit, après `git pull`, le fichier contient des marqueurs :

```
<<<<<<< HEAD
| Membre A | Sara | @sara | |
=======
| Membre B | Omar | @omar | |
>>>>>>> origin/main
```

Gardez les deux lignes, supprimez les trois marqueurs, puis `git add README.md`, `git commit`, `git push`. Un conflit n'est pas une erreur : Git refuse simplement de choisir à votre place.

Contrôle T5 : les deux membres sont comptés

`git shortlog -sn -all` affiche deux noms distincts, puis `python -m pytest tests/test_git_history.py -q` donne **3 passed**.

Si le test annonce `1 auteur(s) de commits, 2 attendus`, deux causes : un membre n'a pas encore poussé, ou les deux postes partagent le même `user.name`. Dans le second cas, corrigez l'identité puis refaites un commit ; l'ancien gardera l'ancien nom, car un commit est immuable.

5.7 Étapes 7 et 8. Vérification et clone frais (15 min)

```
git pull
python scripts\check_setup.py       # T6 : 0 erreur(s), 0 avertissement(s)
python -m pytest -q                 # T6 : 38 passed
```

Les 38 tests se répartissent en 23 d'environnement, 12 de cadrage et 3 d'historique Git. Un seul avertissement reste toléré jusqu'au bout : celui sur la version de Python, si votre poste n'est pas en 3.11. Poussez, puis vérifiez que l'onglet **Actions** est vert.

```
bash scripts/verif_clone.sh https://github.com/votre-compte/sentiment-app.git
```

Contrôle T7 : le dépôt tient debout tout seul

0 erreur(s) et **38 passed** sur un clone frais, sans aucune intervention manuelle : aucun fichier à créer, aucune commande à ajouter, aucun paquet à installer en plus.

Si quelque chose manque ici alors que tout marchait dans votre dossier de travail, c'est qu'un fichier n'a jamais été poussé. C'est exactement ce que fera l'enseignant pour noter.

6 TP2 : entraîner, évaluer, sauvegarder le modèle

Objectif

Produire un modèle reproductible, évalué correctement, et sauvegardé dans un fichier que l'interface du TP3 pourra charger. Pas un bon modèle : un modèle dont vous savez pourquoi il se trompe.

La fiche de cadrage du cas 3, rédigée au TP1, devient le cahier des charges : approche machine learning, métrique de rappel sur la classe négative, seuil d'acceptation.

6.1 Les quatre scripts du kit

Script	Ce qu'il fait	Ce qu'il produit
`src/make_demo.py`	290 avis déterministes : 80 nettement positifs, 80 nettement négatifs, 100 mitigés, 30 neutres	`data/demo_reviews.csv`
`src/train.py`	Découpe, entraîne un pipeline TF-IDF et régression logistique, mesure sur le jeu de test	`models/sentiment.joblib`, `models/metrics.json`
`src/evaluate.py`	Matrice de confusion, rapport de classification, trois erreurs affichées	une lecture, pas un fichier
`src/predict.py`	La fonction `predict()` appelée par l'interface, et plus tard par l'API	l'interface publique du modèle

6.2 Étape A. Installer le kit (15 min)

```
cd tp2-semaine2
python appliquer_tp2.py ../sentiment-app --simulation
python appliquer_tp2.py ../sentiment-app
cd ../sentiment-app && source .venv/bin/activate
python -m pytest -q
```

Contrôle C1 : le point de départ

2 failed, 38 passed, 2 skipped. Les 38 verts sont ceux du TP1, inchangés. Les 2 échecs réclament `models/metrics.json`, qui n'existe pas encore. Les 2 ignorés attendent le modèle lui-même.

Comme au TP1, le rouge au démarrage est voulu : il décrit exactement le travail de la séance.

6.3 Étape B. Le jeu de démonstration (15 min)

```
python src/make_demo.py
```

Contrôle C2 : le jeu est là

290 avis, 145 positifs et 145 négatifs, écrits dans `data/demo_reviews.csv`.

Ouvrez le fichier et lisez trois avis mitigés avant d'aller plus loin. Ils ont tous la même forme :

```
Service impeccable et livraison en avance sur le papier, sauf que ensuite
produit cassé à l'arrivée.                                    -> negative
```

Une première moitié d'un sentiment, un retournement (« mais finalement », « sauf que ensuite »), une seconde moitié de l'autre. L'étiquette suit la **seconde** moitié, celle qui conclut. Un humain lit cela sans hésiter. Retenez cette structure : elle explique chacune des erreurs de l'étape D.

Le jeu est déterministe, graine fixée à 42 : vous et votre binôme obtenez exactement les mêmes 290 avis, donc les mêmes scores. Même principe que le `random_state` du découpage.

6.4 Étape C. Entraîner et sauvegarder (25 min)

```
python src/train.py
```

```
Source : demo, 290 avis (145 négatifs)
232 avis d'entraînement, 58 de test
Vocabulaire : 316 termes
F1 macro sur le jeu de test : 0.879 (seuil 0.85)
Modèle : models/sentiment.joblib
Métriques : models/metrics.json (versionné dans Git, contrairement au modèle)
```

Contrôle C3 : deux fichiers, deux destins

`git status -short` doit montrer `models/metrics.json`, et **pas** `models/sentiment.joblib`, ni `data/demo_reviews.csv`.

La règle est la même pour les deux absents : ce qui se régénère par une commande n'entre pas dans Git ; seule la **mesure** est versionnée. C'est ce qui permettra à l'intégration continue de vérifier la qualité du modèle sans jamais le télécharger (section 9).

6.4.1 Les trois lignes de code à lire

```
X_train, X_test, y_train, y_test = train_test_split(
    df["review"], df["label"], test_size=0.2, random_state=42, stratify=df["label"
        ])
```

Le découpage, avant toute vectorisation. Si vous appreniez le vocabulaire TF-IDF sur l'ensemble des données, les mots du jeu de test influenceraient l'entraînement. Le score monterait, et il serait faux. C'est la fuite de données, l'erreur la plus fréquente et la plus difficile à repérer après coup. `stratify` conserve la proportion des deux classes dans chaque moitié.

```
Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True)),
    ("clf", LogisticRegression(max_iter=1000, class_weight="balanced", random_state
        =42)),
])
```

Le pipeline, un seul objet. Vectoriseur et modèle voyagent ensemble : sauvegarder le classifieur seul donnerait un objet inutilisable, privé de son vocabulaire. C'est aussi ce qui garantit que la transformation appliquée en production est exactement celle de l'entraînement. `ngram_range=(1,2)` ajoute les paires de mots aux mots isolés, `min_df=2` écarte les termes vus une seule fois, `class_weight="balanced"` compense un éventuel déséquilibre des classes.

La graine. `random_state=42` apparaît au découpage et dans le modèle. Sans elle, deux exécutions sur les mêmes données donnent deux scores, et aucune comparaison ne veut plus rien dire.

6.5 Étape D. Évaluer et comprendre les erreurs (30 min)

C'est l'étape la plus importante de la séance. Entraîner prend trois lignes ; savoir ce que vaut le résultat prend une demi-heure.

```
python src/evaluate.py
```

```
              precision    recall  f1-score   support

    negative      0.844     0.931     0.885        29
    positive      0.923     0.828     0.873        29

    accuracy                          0.879        58

Matrice de confusion (lignes = vérité, colonnes = prédiction)
            negative  positive
  negative        27         2
  positive         5        24
```

Contrôle C4 : la matrice de confusion

7 erreurs sur 58, et les trois exemples affichés par le script sont tous des avis mitigés.

La matrice se lit par ligne : sur 29 avis réellement négatifs, 27 ont été reconnus et 2 pris pour des positifs ; sur 29 positifs, 24 reconnus et 5 pris pour des négatifs.

Chiffre	Nom	Ce que ça coûte dans notre cas
27	Vrais négatifs	Avis mécontents correctement remontés
2	Faux positifs	Un client mécontent traité en retard : **le cas qui compte**
5	Faux négatifs	Un avis positif remonté pour rien, sans gravité
24	Vrais positifs	Rien à faire

L'exactitude de 0,879 ne dit rien de tout cela. Ce sont les deux faux positifs qui pèsent, parce que la fiche de cadrage visait le rappel sur la classe négative. **Une métrique unique cache toujours la question qu'on se pose vraiment.**

Pourquoi ces sept erreurs

Le modèle compte des mots positifs et des mots négatifs, puis tranche en faveur du camp le plus fourni. Il ne voit pas le retournement, parce qu'un sac de mots **n'a pas d'ordre** : « bon au début, mauvais ensuite » et « mauvais au début, bon ensuite » lui donnent exactement le même vecteur.

C'est la limite annoncée en cours, et vous venez de la mesurer plutôt que de la croire sur parole. Il faudrait des représentations qui tiennent compte de l'ordre, les embeddings, pour aller plus loin.

Créez `docs/semaine2.md` et répondez en trois phrases : quelles erreurs, pourquoi, et ce qu'il faudrait pour les corriger. C'est noté, et c'est la partie du TP qu'un correcteur lit en premier.

6.6 Étape E. Le vrai jeu, les tests, le push (35 min)

```
python src/train.py --allocine
```

Le script télécharge 20 000 avis de films en français depuis une URL publique (jeu Allociné, 7,6 Mo, quelques secondes) et réentraîne le même pipeline, sans qu'une ligne de code change. Si le réseau est coupé ou filtré, il bascule seul sur le jeu de démonstration et le signale.

	Démonstration	Allociné
Avis	290	20 000
Vocabulaire	316 termes	149 529 termes
F1 macro	0,879	**0,920**
Rappel sur la classe négative	0,931	0,911

Le modèle n'a pas changé : ce sont les données. Un vocabulaire 470 fois plus riche, et surtout une mesure enfin crédible, calculée sur 4 000 avis et non sur 58. Avec 58 avis de test, une erreur de plus déplace le score de deux points : on ne conclut rien d'un tel échantillon.

Contrôle C5 : tout est vert

`python -m pytest -q` affiche **42 passed**, soit les 38 du TP1 plus les 4 du modèle. Poussez, puis vérifiez que l'onglet Actions est vert.

7 TP3 : l'interface de démonstration

Objectif

Rendre le modèle utilisable par quelqu'un qui n'ouvrira jamais un terminal, sans réécrire une ligne de logique de prédiction.

7.1 Streamlit en trois idées

Un script Python devient une page web. Pas de HTML, pas de JavaScript, pas de serveur à configurer : chaque appel à `st.title`, `st.text_area` ou `st.dataframe` ajoute un élément à la page.

Chaque interaction réexécute le script de haut en bas. C'est le modèle de fonctionnement, et il surprend toujours. D'où le décorateur `@st.cache_resource` sur le chargement du modèle : sans lui, le fichier `joblib` serait relu à chaque clic.

C'est une démonstration, pas l'application. Elle sert à montrer le modèle, à récolter des retours, à faire apparaître les cas où il se trompe. C'est l'API, plus tard, qui sera appelée par d'autres programmes.

7.2 Pourquoi deux fichiers

```python
# ui/helpers.py : la logique, testable sans Streamlit
def classer(texte, predicteur):
    resultat = predicteur(texte)
    resultat["incertain"] = resultat["score"] < SEUIL_INCERTITUDE
    resultat["verdict"] = "à vérifier" if resultat["incertain"] else resultat["
        label"]
    return resultat
```

```python
# ui/app.py : l'affichage, et rien d'autre
r = classer(texte, predire)
if r["incertain"]:
    st.warning("À vérifier par un agent")
elif r["label"] == "negative":
    st.error("Avis négatif : à traiter en priorité")
else:
    st.success("Avis positif")
```

Les trois tests du TP3 importent **`helpers.py` sans lancer Streamlit**, et c'est ce que vérifie `test_helpers_importables`. Une interface non testable est une interface qu'on casse sans s'en apercevoir.

7.3 Étape A. Installer et lancer (15 min)

```
cd tp3-streamlit
python appliquer_tp3.py ../sentiment-app
cd ../sentiment-app && source .venv/bin/activate
streamlit run ui/app.py
```

Contrôle S1 : la page s'ouvre

Le navigateur s'ouvre sur `http://localhost:8501` et affiche trois onglets. Si le modèle manque, la page le dit et s'arrête : lancez `python src/train.py -allocine`, puis rechargez.

Ce terminal reste occupé tant que l'interface tourne : ouvrez-en un second pour la suite, et arrêtez avec `Ctrl+C`.

7.4 Étape B. L'onglet « un avis » (30 min)

Saisissez trois avis : un clairement positif, un clairement négatif, puis un mitigé du type « service impeccable au début, mais finalement produit cassé ».

Contrôle S2 : le modèle dit quand il hésite

L'avis mitigé doit afficher « **à vérifier** » plutôt qu'un verdict tranché, dès que la confiance tombe sous 60 %.

Le seuil est dans `ui/helpers.py`, constante `SEUIL_INCERTITUDE`. Faites-le varier et notez ce qui change :

Seuil	Avis envoyés à un agent	Risque
0,50	peu	Le modèle tranche seul des cas qu'il ne maîtrise pas
0,60	modéré	Compromis retenu par défaut
0,75	beaucoup	La file humaine se remplit, l'automatisation perd son intérêt

Ce curseur ne se règle pas dans le code : il se décide avec l'équipe métier, et il découle directement de la fiche du cas 3, qui disait qu'un avis mal classé se requalifie d'un clic.

7.5 Étape C. L'onglet « un fichier CSV » (40 min)

Téléversez `data/sample_reviews.csv`, puis `data/demo_reviews.csv` produit au TP2.

Contrôle S3 : le résumé et le tableau

Trois métriques en haut (avis analysés, négatifs détectés avec leur pourcentage, cas à vérifier), un graphique de répartition, un tableau trié négatifs en tête, et un bouton de téléchargement.

Sur `demo_reviews.csv`, comparez la part de négatifs annoncée par l'interface avec le vrai étiquetage du fichier : l'écart correspond exactement aux erreurs mesurées au TP2.

7.6 Étape D. Tests, documentation, push (35 min)

```
python -m pytest tests/test_ui.py -q      # 3 passed
python -m pytest -q                       # 45 passed
git add -A
git commit -m "Semaine 3 : interface de demonstration Streamlit"
git push
```

Contrôle S4 : tout est vert

45 passed, soit 38 du TP1, 4 du TP2 et 3 du TP3, puis la CI verte.

Écrivez `docs/semaine3.md`, trois phrases : quel seuil vous retenez, combien d'avis cela envoie à un agent sur le jeu de démonstration, et pourquoi ce compromis vous paraît juste.

7.7 Sur Google Colab

Streamlit ne s'affiche pas directement dans Colab : le serveur tourne sur la machine virtuelle, que votre navigateur ne peut pas joindre. Le notebook `TP3-colab-streamlit.ipynb` passe donc par un tunnel.

```
!curl -s https://loca.lt/mytunnelpassword      # le mot de passe du tunnel : l'IP
    publique
!streamlit run ui/app.py --server.headless true --server.port 8501 &>/content/log &
!npx --yes localtunnel --port 8501             # affiche une adresse loca.lt à
    ouvrir
```

Trois différences avec un poste local

Le tunnel donne une adresse **publique et temporaire** : aucune donnée réelle ne doit y transiter.

Pas d'environnement virtuel : Colab installe dans son Python système, donc `test_virtualenv_active` est écarté et les comptes deviennent 37, 41 et 44 au lieu de 38, 42 et 45.

Rien ne survit à la session : `/content` disparaît à la fermeture. Poussez sur GitHub avant de partir. Le modèle, lui, n'a pas à être conservé.

8 Les 45 tests

Objectif

Comprendre ce que la machine vérifie, pour corriger seul au lieu d'attendre.

Le dépôt se vérifie de deux façons, qui contrôlent les mêmes choses. `scripts/check_setup.py` parle aux humains, avec des lignes OK, ATTENTION, ERREUR. `pytest` parle aux machines, et c'est lui que la CI et la notation utilisent.

8.1 Répartition

Fichier	Tests	Ce qu'il vérifie
`test_environment.py`	23	Version Python, venv actif, trois paquets, sept dossiers, quatre fichiers, versions figées, cinq motifs du `.gitignore`, rien de secret commité
`test_cadrage.py`	12	Trois fiches, huit rubriques, rubriques remplies, une seule case cochée, deux critères cités
`test_git_history.py`	3	Au moins un commit, deux auteurs distincts, `.venv/` absent de tout l'historique
`test_model.py`	4	Métriques complètes, F1 au-dessus du seuil, prédiction correcte, format de sortie
`test_ui.py`	3	Helpers importables sans Streamlit, verdict ajouté, traitement d'un lot

8.2 Ce qui est ignoré, et pourquoi

Certains tests s'effacent au lieu d'échouer quand leur condition d'exécution n'est pas réunie. C'est voulu, et cela explique les `skipped` des contrôles T1 et C1.

Tests	Ignorés quand
Les 3 de `test_git_history.py`	Git manque, `.git/` n'existe pas, ou aucun commit
`test_no_real_env_file_committed`	Idem
Les 2 tests de prédiction du modèle	Le dossier n'est pas un dépôt Git
Les 2 tests d'interface utilisant le modèle	`models/sentiment.joblib` absent, ce qui est le cas en CI

8.3 Les paramètres réglables

Paramètre	Fichier	Valeur	Rôle
`MIN_AUTHORS`	`check_setup.py`, `test_git_history.py`	2	Auteurs exigés
`REQUIRED_DIRS`	`check_setup.py`, `test_environment.py`	7	La structure attendue
`REQUIRED_SECTIONS`	`test_cadrage.py`	8	Rubriques des fiches
`CRITERIA`	`test_cadrage.py`	(a) à (d)	Lettres acceptées
`SEUIL_F1`	`train.py`, `test_model.py`	0,85	Seuil d'acceptation
`SEUIL_INCERTITUDE`	`ui/helpers.py`	0,60	Bascule vers « à vérifier »

Ces constantes sont en haut de chaque fichier, lisibles sans chercher. **Ne les modifiez pas pour faire passer un test** : c'est exactement ce que la notation regarde. Le seul qu'on vous demande de faire varier est `SEUIL_INCERTITUDE`, et c'est l'objet du contrôle S2.

9 Intégration continue et MLOps

9.1 Trois artefacts, pas un

Un logiciel classique versionne une chose : le code. Une application intelligente en versionne trois, et ce tableau est la règle qui décide, pour chacune, où elle vit.

Artefact	Dans Git ?	Pourquoi
Le code (`src/`, `ui/`, `tests/`)	Oui	C'est le travail
L'environnement (`.venv/`)	Non	Reconstructible : `pip install -r requirements.txt`
Les données (`demo_reviews.csv`)	Non	Reconstructible : `python src/make_demo.py`
Le modèle (`sentiment.joblib`)	Non	Reconstructible : `python src/train.py`
La mesure (`metrics.json`)	**Oui**	Elle ne se reconstruit pas sans réentraîner

9.2 Ce que fait l'intégration continue

```yaml
on: [push, pull_request]
jobs:
  tp1:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: python -m venv .venv && . .venv/bin/activate && pip install pandas
          scikit-learn pytest
      - run: . .venv/bin/activate && python scripts/check_setup.py
      - run: . .venv/bin/activate && pytest -q
```

Paramètre	Rôle
`on: [push, pull_request]`	Les tests tournent à chaque push et à chaque pull request
`fetch-depth: 0`	Récupère **tout l'historique**. Par défaut Actions ne clone qu'un commit, et `git shortlog` ne verrait qu'un auteur : le test échouerait toujours
`python-version: "3.11"`	La version de référence du cours, quelle que soit la vôtre
Création du venv dans chaque `run`	Sinon `test_virtualenv_active` échouerait
Trois paquets seulement	La CI dure une minute ; l'installation complète est vérifiée au clone frais

Une CI verte veut dire que votre dépôt fonctionne **ailleurs que sur vos deux portables**. Si elle est rouge alors que tout passe chez vous, ouvrez le journal : c'est le même pytest, et le nom du test en échec dit quoi corriger.

9.3 Le geste MLOps de la séquence

La CI n'a jamais votre modèle

`models/sentiment.joblib` n'est pas dans le dépôt. Pourtant, l'intégration continue vérifie qu'un modèle acceptable a été produit.

Les 4 tests du modèle	En intégration continue
Deux lisent `models/metrics.json`	Ils passent : le fichier est versionné
Deux appellent `predict()`	Ils sont ignorés : le modèle est absent

On transporte la **preuve** de la qualité, pas l'artefact lourd. C'est le principe que MLflow et DVC industrialiseront plus tard dans le semestre, et la porte de validation qui déclenchera le réentraînement automatique quand la dérive sera au programme.

9.4 DevOps, MLOps : où passe la frontière

DevOps, dans ces trois TP	MLOps, ce qui s'y ajoute
Environnement reproductible	Trois artefacts au lieu d'un
Tests automatisés	Une validation qui compare une performance à un seuil, au lieu de passer ou d'échouer
Versionnage à plusieurs	Des données et un modèle à versionner, trop lourds pour Git
Intégration continue	Une dégradation possible **sans qu'on ait rien modifié** : la dérive

Un test unitaire passe ou échoue. Un modèle a un F1 à comparer à un seuil, sur un jeu de test figé : c'est pour cela que `metrics.json` existe, et que le seuil y est inscrit à côté de la mesure.

10 Le compte rendu global

À la fin des trois séances, chaque binôme rédige un compte rendu unique, de quatre à six pages, déposé dans son dépôt et figé par une étiquette Git.

```
cp docs/compte-rendu-TEMPLATE.md docs/compte-rendu.md
# rédaction...
git add docs/compte-rendu.md
git commit -m "Compte rendu global des TP1 a TP3"
git push
git tag -a compte-rendu -m "Version remise le <date>"
git push origin compte-rendu
```

L'étiquette fige la version remise : les commits postérieurs ne sont pas pris en compte. Format Markdown, dans le dépôt, pas de PDF ni de lien externe : le compte rendu vit avec le code qu'il décrit.

Section	Pts	Ce qui est regardé
1. Le besoin et son cadrage	3	Les lettres (a) à (d) comme raisonnement, pas comme formalité
2. L'architecture du dépôt	3	Vous savez dire pourquoi `metrics.json` est dans Git et pas le modèle
3. Le modèle	4	Vos F1 correspondent à votre `metrics.json` ; le découpage avant vectorisation est expliqué
4. L'analyse des erreurs	4	Les avis mitigés et l'absence d'ordre, avec un exemple tiré de **vos** données
5. Le seuil d'incertitude	3	Combien d'avis partent vers un agent, et pourquoi ce compromis
6 à 8. Échecs, recul, répartition	3	Un compte rendu sans aucun échec n'est pas crédible

Trois règles : citez **vos** chiffres et non ceux du corrigé, avec une phrase sur l'écart le cas échéant ; un seul compte rendu par binôme, avec une section sincère sur la répartition du travail ; et si vous utilisez un assistant d'IA pour rédiger, dites-le en une phrase, en précisant ce que vous avez vérifié vous-même. C'est autorisé, le cacher ne l'est pas.

11 Récapitulatif

11.1 Les dix-sept contrôles

	Commande	Résultat attendu
T0	`ls` et `ls -a`	7 dossiers, `.gitignore` présent
T1	`pytest -q`	3 failed, 31 passed, 4 skipped
T2	`pytest tests/test_cadrage.py -q`	12 passed
T3	`git status` avant le commit	aucune ligne `.venv/`
T4	`python scripts/check_setup.py`	0 erreur(s)
T5	`git shortlog -sn -all` puis les tests Git	deux noms, 3 passed
T6	diagnostic et `pytest -q`	0 erreur, 0 avertissement, 38 passed
T7	la suite sur un clone frais	38 passed sans intervention
C1	`pytest -q` après le kit TP2	2 failed, 38 passed, 2 skipped
C2	`python src/make_demo.py`	290 avis, 145 de chaque classe
C3	`python src/train.py`	F1 macro 0,879 ; seul `metrics.json` en attente
C4	`python src/evaluate.py`	7 erreurs sur 58, toutes sur des avis mitigés
C5	`train.py -allocine` puis `pytest -q`	F1 0,920 ; 42 passed ; CI verte
S1	`streamlit run ui/app.py`	la page s'ouvre, trois onglets
S2	un avis mitigé	verdict « à vérifier » sous 60 %
S3	un CSV dans l'onglet 2	métriques, graphique, tableau trié, export
S4	`pytest -q`	45 passed, CI verte

11.2 Les erreurs les plus coûteuses

Message	Cause	Correction
`pytest n'est pas reconnu`	Venv non activé	Activer, ou `python -m pytest`
`Activate.ps1 cannot be loaded`	Politique PowerShell	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`ModuleNotFoundError`	Venv non activé ou installation partielle	`where python` doit contenir `.venv`
`No matching distribution found`	Python 3.12 ou 3.13	Installer 3.11, recréer le venv
`error: 400` sur l'URL	Chevrons restés dans l'adresse	`git remote set-url origin <vraie URL>`
`Password authentication is not supported`	Authentification	Jeton d'accès personnel
`403 Forbidden`	Autorisation	Accepter l'invitation de collaborateur
`rejected, non-fast-forward`	L'autre membre a poussé	`git pull`, résoudre, `git push`
`1 auteur(s) de commits, 2 attendus`	Même `user.name` sur les deux postes	Identité distincte, puis un vrai commit
`.venv/ a Ã¯tÃ¯ commitÃ¯`	`.gitignore` absent au premier `add`	Refaire le dépôt
`models/metrics.json absent`	Étape C du TP2 sautée	`python src/train.py`
`models/sentiment.joblib absent`	Modèle non entraîné sur ce poste	`python src/train.py -allocine`

Message	Cause	Correction
`Port 8501 is already in use`	Une interface tourne déjà	`Ctrl+C`, ou `-server.port 8502`

11.3 La structure finale du dépôt

```
sentiment-app/
|-- README.md TP1-ENONCE.md MARCHE-A-SUIVRE.md TESTS.md AIDE-GIT.md DEPANNAGE.md
|-- requirements.txt requirements.lock .gitignore .env.example pytest.ini Makefile
|-- .github/workflows/ci.yml        intégration continue
|-- docs/
|   |-- cadrage/cas1, cas2, cas3    TP1, remplies (livrable noté)
|   |-- semaine2.md                 TP2, analyse des erreurs
|   |-- semaine3.md                 TP3, seuil d'incertitude
|   |-- compte-rendu.md             le rendu final, figé par une étiquette Git
|-- src/       make_demo, train, evaluate, predict          (TP2)
|-- ui/        app.py, helpers.py                           (TP3)
|-- tests/     environment (23), cadrage (12), git (3), model (4), ui (3)
|-- scripts/   check_setup, setup, init_depot, verif_clone, new_cadrage
|-- models/    .gitkeep + metrics.json    le modèle n'est JAMAIS commité
|-- data/      sample_reviews.csv         demo_reviews.csv est régénérable
```

45 tests à l'arrivée. Trois artefacts restent hors de Git parce qu'ils se reconstruisent en une commande : le venv, le jeu de démonstration, le modèle. Une seule mesure est versionnée. C'est le principe qui portera tout le reste du semestre, et le dépôt d'aujourd'hui est celui de la dernière semaine.